Programming in c [ppc18]

Unit-1

Introduction to computers

By

Shobha.K

CSE(Cyber Security)

Ramaiah Institute Of Technology Bangalore

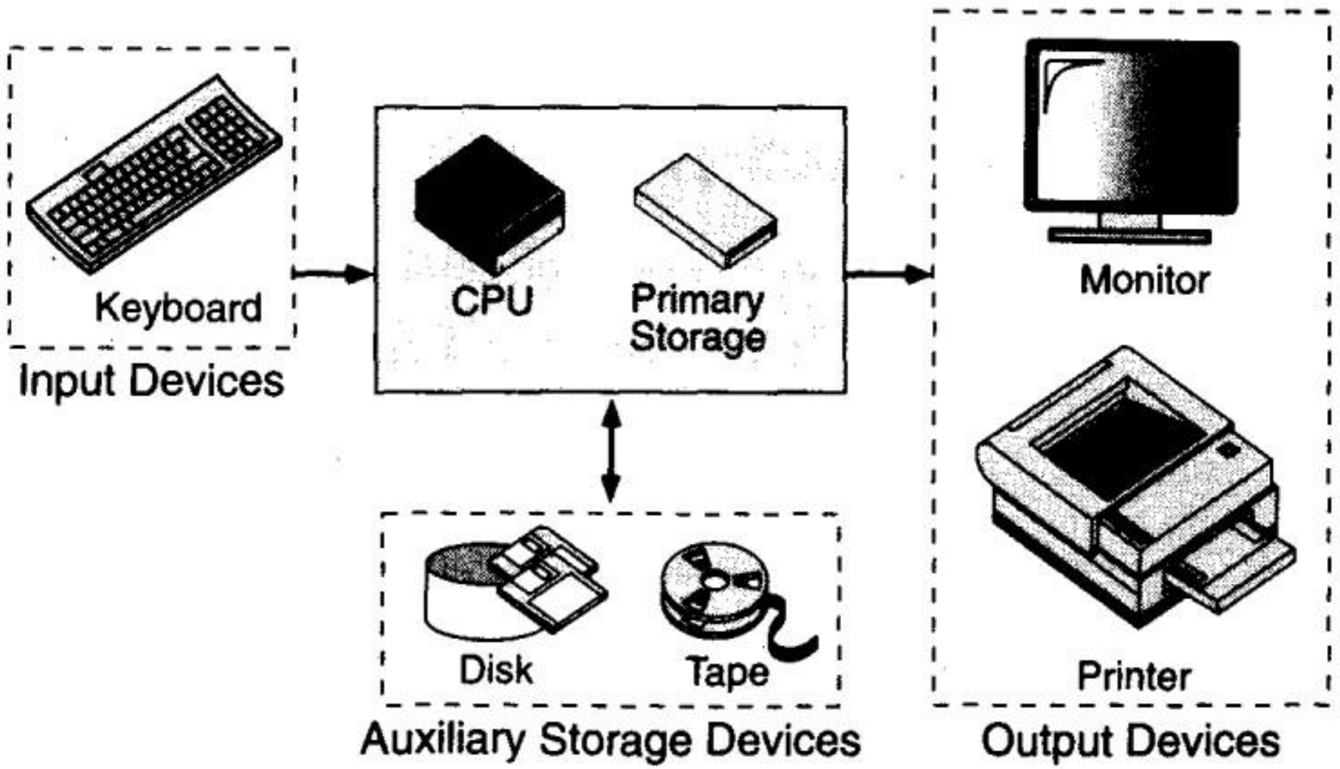
Computer Systems:

- Computer system is a programmable Electronic device that can accept Input,store data & retrieve ,process & output Information.
- Computer is a System made of 2 major components
    1. Hardware
    2. Software
- The Computer Hardware is physical Equipment.
- The Software is a collection of programs (instructions) that allows hardware to do its job

Computer Hardware

- The hardware component of the computer system consists of five parts: input devices, central processing unit (CPU) ,primary storage, output devices, and auxiliary storage devices.

Computer Hardware



Basic Hardware Components

Contd..

- The **input device** is usually a keyboard where programs and data are entered into the computers. Examples of other input devices include a mouse, a pen or stylus, a touch screen, or an audio input unit.
- The **central processing unit (CPU**) is responsible for executing instructions such as arithmetic calculations, comparisons among data, and movement of data inside the system. Today's computers may have one ,two, or more CPUs .
- **Primary storage** ,also known as main memory, is a place where the programs and data are stored temporarily during processing. The data in primary storage are erased when we turn off a personal computer or when we log off from a time-sharing system.
- The **output device** is usually a monitor or a printer to show output. If the output is shown on the monitor, we say we have a **soft copy**. If it is printed on the printer, we say we have a hard copy.
- **Auxiliary storage**, also known as **secondary storage**, is used for both input and output. It is the place where the programs and data are stored permanently. When we turn off the computer, or programs and data remain in the secondary storage, ready for the next time we need them.

Computer Software

- Computer software is divided in to two broad categories: system software and application software .System software manages the computer resources .It provides the interface between the hardware and the users. Application software, on the other hand is directly responsible for helping users solve their problems

Types Of Computer Software

- System Software
- Application Software

System Software:

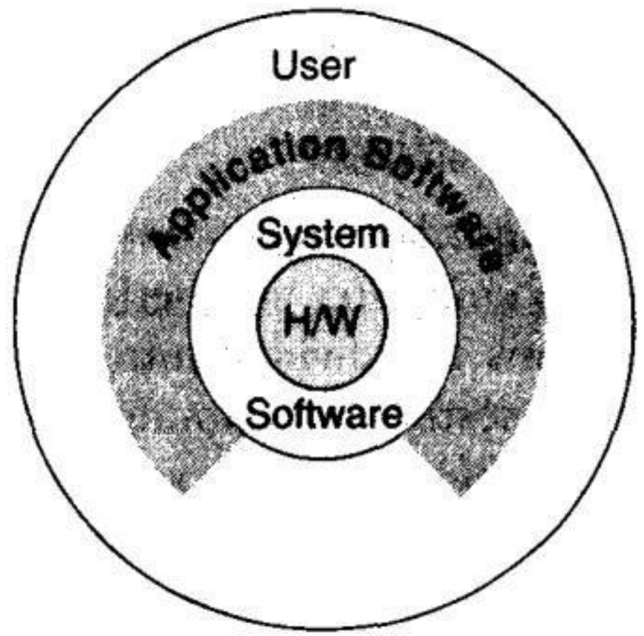
- **System software** consists of programs that manage the hardware resources of a computer and perform required information processing tasks. These programs are divided into three classes: the operating system, system support, and system development.
- The **operating system** provides services such as a user interface, file and database access, and interfaces to communication systems such as Internet protocols. The primary purpose of this software is to keep the system operating in an efficient manner while allowing the users access to the system.
- **System support software** provides system utilities and other operating services. Examples of system utilities are sort programs and disk format programs. Operating services consists of programs that provide performance statistics for the operational staff and security monitors to protect the system and data.
- The last system software category ,**system development software**, includes the language translators that convert programs into machine language for execution ,debugging tools to ensure that the programs are error free and computer –assisted software engineering(CASE) systems.

Application software

- **Application software** is broken in to two classes :general-purpose software and application – specific software. **General purpose software** is purchased from a software developer and can be used for more than one application. Examples of general purpose software include word processors ,database management systems ,and computer aided design systems. They are labeled general purpose because they can solve a variety of user computing problems.
- **Application –specific software** can be used only for its intended purpose.

- A general ledger system used by accountants and a material requirements planning system used by a manufacturing organization are examples of application-specific software. They can be used only for the task for which they were designed they cannot be used for other generalized tasks.
- The relationship between system and application software is shown in fig.In this figure, each circle represents an interface point .The inner core is hard ware. The user is represented by the out layer. To work with the system, the typical user uses some form of application software. The application software in turn interacts with the operating system , which is apart of the system software layer. The system software provides the direct interaction with the hard ware. The opening at the bottom of the figure is the path followed by the user who interacts directly with the operating system when necessary.

Relationship b/n system and Application Software



Relationship Between System and Application Software

Computer Languages:

- To write a program for a computer, we must use a **computer language.** Over the years computer languages have evolved from machine languages to natural languages.
- 1940’s Machine level Languages
- 1950’s Symbolic Languages
- 1960’s High-Level Languages

Machine Languages

- In the earliest days of computers, the only programming languages available were machine languages. Each computer has its own machine language, which is made of streams of 0’s and 1’s.
- Instructions in machine language must be in streams of 0’s and 1’s because the internal circuits of a computer are made of switches transistors and other electronic devices that can be in one of two states: off or on. The off state is represented by 0 , the on state is represented by 1.
- The only language understood by computer hardware is machine language.

Symbolic Languages:

- In early 1950's Admiral Grace Hopper, A mathematician and naval officer developed the concept of a special computer program that would convert programs into machine language.
-

  The early programming languages simply mirror to the machine languages using symbols of mnemonics to represent the various machine language instructions because they used symbols, these languages were known as symbolic languages.
- Computer does not understand symbolic language it must be translated to the machine language. A special program called assembler translates symbolic code into machine language. Because symbolic languages had to be assembled into machine language they soon became known as assembly languages.
- Symbolic language uses symbols or mnemonics to represent the various ,machine language instructions.

High Level Languages:

- Symbolic languages greatly improved programming efficiency; they still required programmers to concentrate on the hardware that they were using. Working with symbolic languages was also very tedious because each machine instruction has to be individually coded. The desire to improve programmer efficiency and to change the focus from the computer to the problem being solved led to the development of high-level language.
- High level languages are portable to many different computers, allowing the programmer to concentrate on the application problem at hand rather than the intricacies of the computer. High-level languages are designed to relieve the programmer from the details of the assembly language. High level languages share one thing with symbolic languages, They must be converted into machine language. The process of converting them is known as compilation.
- The first widely used high-level languages, FORTRAN (Formula Translation)was created by John Backus and an IBM team in 1957;it is still widely used today in scientific and engineering applications. After FORTRAN was COBOL(Common Business-Oriented Language). Admiral Hopper was played a key role in the development of the COBOL Business language.
- **C is a high-level language used for system software and new application code.**

Creating & Running Programs

- It is the job of Programmer to write & test the Program. There are 4 steps in this Process.

1. Writing & Editing the program
2. Compiling the program
3. Linking the program with required library modules
4. Executing the Program

- For C Programs use text Editor to write the program and save program with filename.c.
- For compiling use CC filename.c
- For Executing use ./a.out

Computing Environment

- **Definition:**
- Computing environment is a collection of computers/machines, software and networks that support the processing and exchange of electronic information.
  Different types of computing environments
- There are **four types** of computing environments. They are:
- Personal computing environment
- Time-Sharing environment
- Client -Server environment
- Distributed environment

Personal Computing Environment:

- A personal computer(PC) is a general purpose computer. The size and capabilities of a personal computer makes it useful for end-users.
- In a personal computer, all the hardware components are tied together, so that it can be used as per our need.

Time Sharing Environment

- Employees in large companies often work in a time sharing environment. In this environment, many users are connected to one or more computers.
- These are minicomputers or central mainframes.
- The terminals used are not programmable, but now we see more microcomputers that are used to simulate the environments terminals.
- In the time sharing environment, the output devices and auxiliary storage are shared by all the users.
- In time sharing environment, each computer must be operated by the central computer.
- In this type of environment, computing is operated by central computer. This central computer has many duties which keeps the computer busy.

Time Sharing Environment

Client Server Environment

- Client/Server is a distribute computing model, in which the client requests services from the server.
- Client/Server usually run on separate computers connected by a network.
- A client is a process or an application that sends messages to a server through the network.
- These messages ask the server to perform a specific task, like finding a customer record in a database.
- Severs usually run on powerful computers, workstations or mainframes
- The administration of the installed equipment is more expensive than maintaining a centralized system

Contd..


Server
Central Storage
Shared
Printers
Client 1
Client 2
Client 3
Client 4
Client-Server Environment

Distributed Computing Environment

- A distributed computing environment allows smooth integration of computing functions between different internet server's clients and provides the connection to different servers around the world.
- Distributed Computing is used by distributed systems to solve computational problems
- In Distributed Computing, each problem is divided into several tasks, which are solved by one or more computers.

Contd..

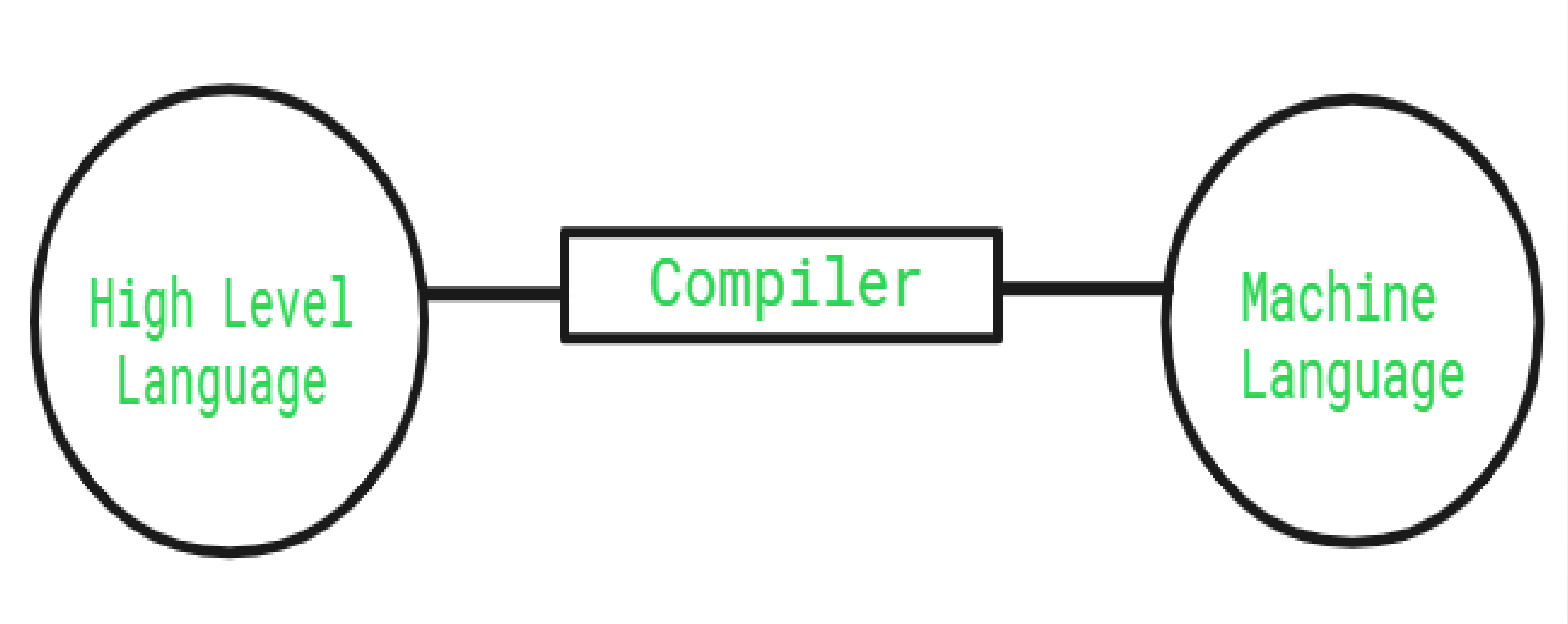
Distributed Computing Environment

Compiler & Interpreter

- The Compiler is a translator which takes input i.e., High-Level Language, and produces an output of low-level language i.e. machine or assembly language. The work of a Compiler is to transform the codes written in the programming language into machine code (format of 0s and 1s) so that computers can understand.
- A compiler is more intelligent than an assembler it checks all kinds of limits, ranges, errors, etc.
- But its program run time is more and occupies a larger part of memory. It has a slow speed because a compiler goes through the entire program and then translates the entire program into machine codes.

Role of a Compiler

- For Converting the code written in a high-level language into machine-level language so that computers can easily understand, we use a compiler. Converts basically convert high-level language to intermediate assembly language by a compiler and then assembled into machine code

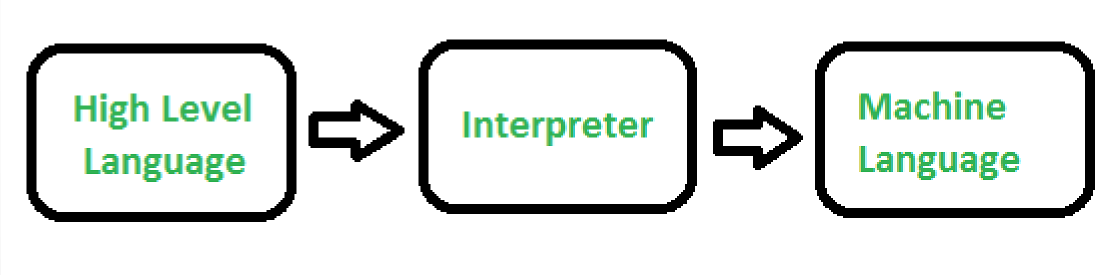


Adv & Disadv of Compiler

- **Advantages of Compiler**
- Compiled code runs faster in comparison to Interpreted code.
- Compilers help in improving the security of Applications.
- As Compilers give Debugging tools, which help in fixing errors easily.
- **Disadvantages of Compiler**
- The compiler can catch only syntax errors and some semantic errors.
- Compilation can take more time in the case of bulky code.

Interpreter

- An Interpreter is a program that translates a programming language into a comprehensible language. The interpreter converts high-level language to an intermediate language. It contains pre-compiled code, source code, etc.
- It translates only one statement of the program at a time.
- Interpreters, more often than not are smaller than compilers.
- **Role of an Interpreter**
- The simple role of an interpreter is to translate the material into a target language. An Interpreter works line by line on a code. It also converts high-level language to machine language.
-



Adv & Disadv of Interpreter

- **Advantages of Interpreter**
- Programs written in an Interpreted language are easier to debug.
- Interpreters allow the management of memory automatically, which reduces memory error risks.
- Interpreted Language is more flexible than a Compiled language.
- **Disadvantages of Interpreter**
- The interpreter can run only the corresponding Interpreted program.
- Interpreted code runs slower in comparison to Compiled code.

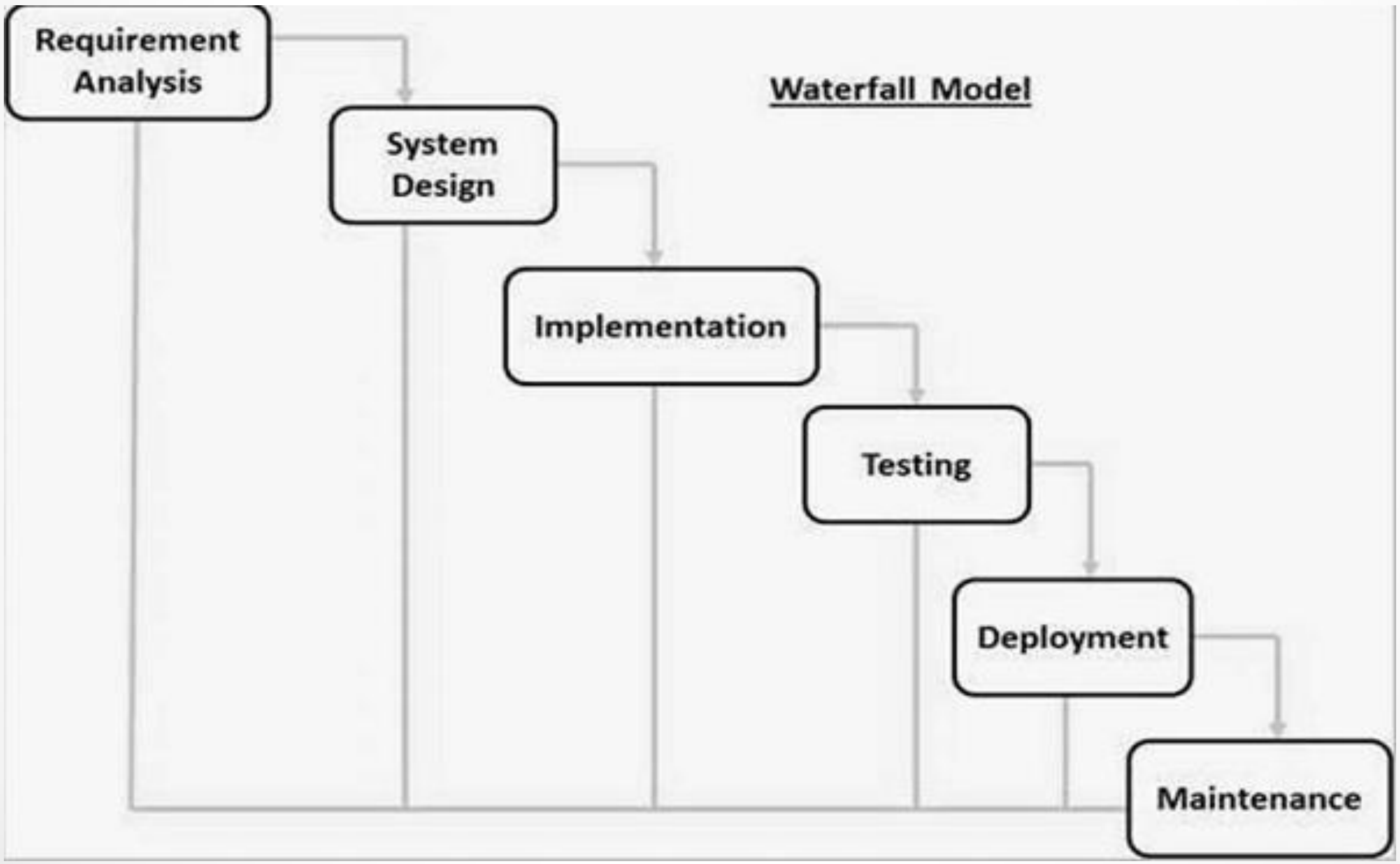
Difference Between Compiler and Interpreter

Compiler	Interpreter
The compiler saves the Machine Language in form of Machine Code on disks.	The Interpreter does not save the Machine Language.
Compiled codes run faster than Interpreter.	Interpreted codes run slower than Compiler.
Linking-Loading Model is the basic working model of the Compiler.	The Interpretation Model is the basic working model of the Interpreter.
The compiler generates an output in the form of (.exe).	The interpreter does not generate any output.
Any change in the source program after the compilation requires recompiling the entire code.	Any change in the source program during the translation does not require retranslation of the entire code.
Errors are displayed in Compiler after Compiling together at the current time.	Errors are displayed in every single line.

System Development

- **System Development Life Cycle**
- Waterfall Model – Design
- Waterfall approach was first SDLC Model to be used widely in Software Engineering to ensure success of the project. In "The Waterfall" approach, the whole process of software development is divided into separate phases. In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.
- The following illustration is a representation of the different phases of the Waterfall Model.

Contd..


Requirement Analysis
Waterfall Model
System Design
Implementation
Testing
Deployment
Maintenance

Contd..

- **Requirement Gathering and analysis** – All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- **System Design** – The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- **Implementation** – With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.
- **Integration and Testing** – All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
- **Deployment of system** – Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
- **Maintenance** – There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

Program Development &Testing

- Understanding the problem, Develop a solution, Write the Program and then test it.

Testing Contains 2 types

- Black Box Testing: Testing the program without knowing how it works and what is inside it
- White Box Testing: Assumes that Tester knows everything about the program.

Introduction to C Programming

C was first designed by Dennis Ritchie for use with UNIX on DEC PDP-11 computers. The language evolved from Martin Richard's BCPL, and one of its earlier forms was the B language, which was written by Ken Thompson for the DEC PDP-7. The first book on C was The C Programming Language by Brian Kernighan and Dennis Ritchie, published in 1978.

In 1983, the American National Standards Institute (ANSI) established a committee to standardize the definition of C. The resulting standard is known as ANSI C, and it is the recognized standard for the language, grammar, and a core set of libraries. The syntax is slightly different from the original C language, which is frequently called K&R for Kernighan and Ritchie. There is also an ISO (International Standards Organization) standard that is very similar to the ANSI standard.

BASIC STRUCTURE OF C LANGUAGE:

- The program written in C language follows this basic structure. The sequence of sections should be as they are in the basic structure. A C program should have one or more sections but the sequence of sections is to be followed.

1. Documentation section

2.Linking section

3.Definition section

4.Global declaration section

5.Main function section

{

Declaration section Executable section

}

6.Sub program or function section

Contd..

- **DOCUMENTATION SECTION:** comes first and is used to document the use of logic or reasons in your program. It can be used to write the program's objective, developer and logic details. The documentation is done in C language with /* and */ . Whatever is written between these two are called comments.
- **LINKING SECTION :** This section tells the compiler to link the certain occurrences of keywords or functions in your program to the header files specified in this section.
- e.g. #include <stdio.h>

- **DEFINITION SECTION :** It is used to declare some constants and assign them some value.

- e.g. #define MAX 25

- Here #define is a compiler directive which tells the compiler whenever MAX is found in the program replace it with 25.

Contd..

- **GLOBAL DECLARATION SECTION :** Here the variables which are used through out the program (including main and other functions) are declared so as to make them global(i.e accessible to all parts of program)
- e.g. int i; (before main())
- **MAIN FUNCTION SECTION :** It tells the compiler where to start the execution from main()

{

point from execution starts

}

- main function has two sections

declaration section : In this the variables and their data types are declared.

Executable section : This has the part of program which actually performs the task we need.

- **SUB PROGRAM OR FUNCTION SECTION :** This has all the sub programs or the functions which our program needs.

Sample Program

```
#include<stdio.h>
 int main()
        {
          printf("Hello");
        }
```

Output:

Hello

Program for sum of 3 numbers

```
#include<stdio.h>
 int main()
  {
    int a,b,c,sum;
    printf("enter the values:\n");
    scanf("%d%d%d",&a,&b,&c);
     sum=a+b+c;
     printf("sum of numbers =%d",sum);
      return 0;
  }
```

Output:enter values: 2   3   4

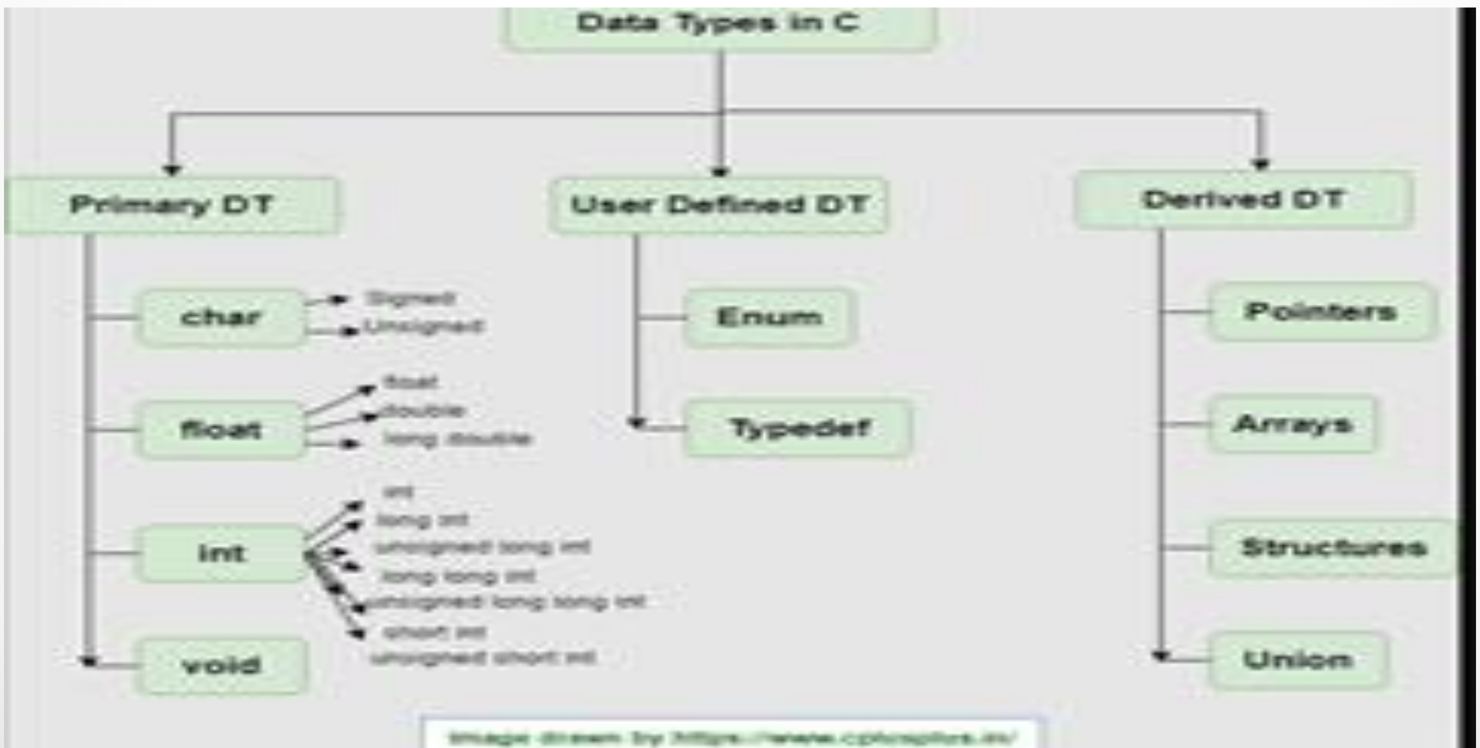
Sum of number=9

IDENTIFIERS

- Names of the variables and other program elements such as functions, array,etc,are known as identifiers.
- There are few rules that govern the way variable are named(identifiers).
- Identifiers can be named from the combination of A-Z, a-z, 0-9, _(Underscore).
- The first alphabet of the identifier should be either an alphabet or an underscore. digit are not allowed.
- It should not be a keyword. Eg: name,ptr,sum

Data Type

- To represent different types of data in C program we need different data types. A data type is essential to identify the storage representation and the type of operations that can be performed on that data. C supports four different classes of data types namely



Example for Integer and float data type

1).Example for Integer data type

```
#include<stdio.h>
int main()
{
  int a,b,sum;
 printf(“enter the values for a & b”);
 scanf(“%d%d”,&a,&b);
sum=a+b;
printf(“sum of a & b=%d”,sum);
return 0;
}
```

2)Example for float data type

```
#include<stdio.h>
int main()
{
  float a,b,sum;
 printf(“enter the values for a & b”);
 scanf(“%f%f”,&a,&b);
sum=a+b;
printf(“sum of a & b=%f”,sum);
return 0;
}
```

Example for double and character

- **Example for Double data type**

```
#include<stdio.h>
int main()
{
  double a,b,sum;
 printf("enter the values for a & b");
 scanf("%lf%lf",&a,&b);
sum=a+b;
printf("sum of a & b=%lf",sum);
return 0;
}
```

Example for Character Data Type:

```
#include<stdio.h>
Int main()
{
char a;
printf(" chracter a="%c",a);
return 0;
}
```

Example of Arrays

```
#include<stdio.h>
int main()
{
  int a[10],n;
 printf("enter the size of the array");
 scanf("%d",&n);
 printf("enter the elements"):
for(i=0;i<n;i++){
scanf("%d",&a[i]);
printf("The elements of the array=%d",a[i]);
return 0;
}
```

How to define Structure/Union

```
struct Student
{
   char name[10];
    int phone;
   int rollno;
    float fees;
};
Stud[10];
```

For union also same way only change struct to union

Typedef And Enum

Example for Enum:

```
#include<stdio.h>
enum week{“sun,mon,tue,wed,thu,fri,sat};
 int main()
{
 enum week today;
 today=wed;
 printf(“Day is %d”,today+1);
}
```

Example for typedef:

```
#include<stdio.h>
Typedef char user[10];
int main()
{
 user  user1=“nithin”;
 return 0;
}
```

Typedef gives an existing data type an alias name.

ASSIGNING VALUES :

- /* write a program to show assigning of values to variables */

```
#include<stdio.h>
main()

{


int a; float b;
printf("Enter any number\n");
b=190.5;
scanf("%d",&a);

printf("user entered %d", a);

printf("B's values is %f", b);

}
```

Formatted Input and Formatted Output

- The simplest of input operator is getchar to read a single character from the input device.

- varname=getchar(); you need to declare varname.

The simplest of output operator is putchar to output a single character on the output device.

putchar(varname)

- The getchar() is used only for one input and is not formatted. Formatted input refers to an input data that has been arranged in a particular format, for that we have scanf.
- scanf("control string", arg1, arg2,...argn);

Contd..

- Control string specifies field format in which data is to be entered.
- arg1, arg2... argn specifies address of location or variable where data is stored.
- eg scanf("%d%d",&a,&b);
- %d used for integers
- %f floats
- %l long
- %c character for formatted output you use printf
- printf("control string", arg1, arg2,...argn);

program to exhibit i/o

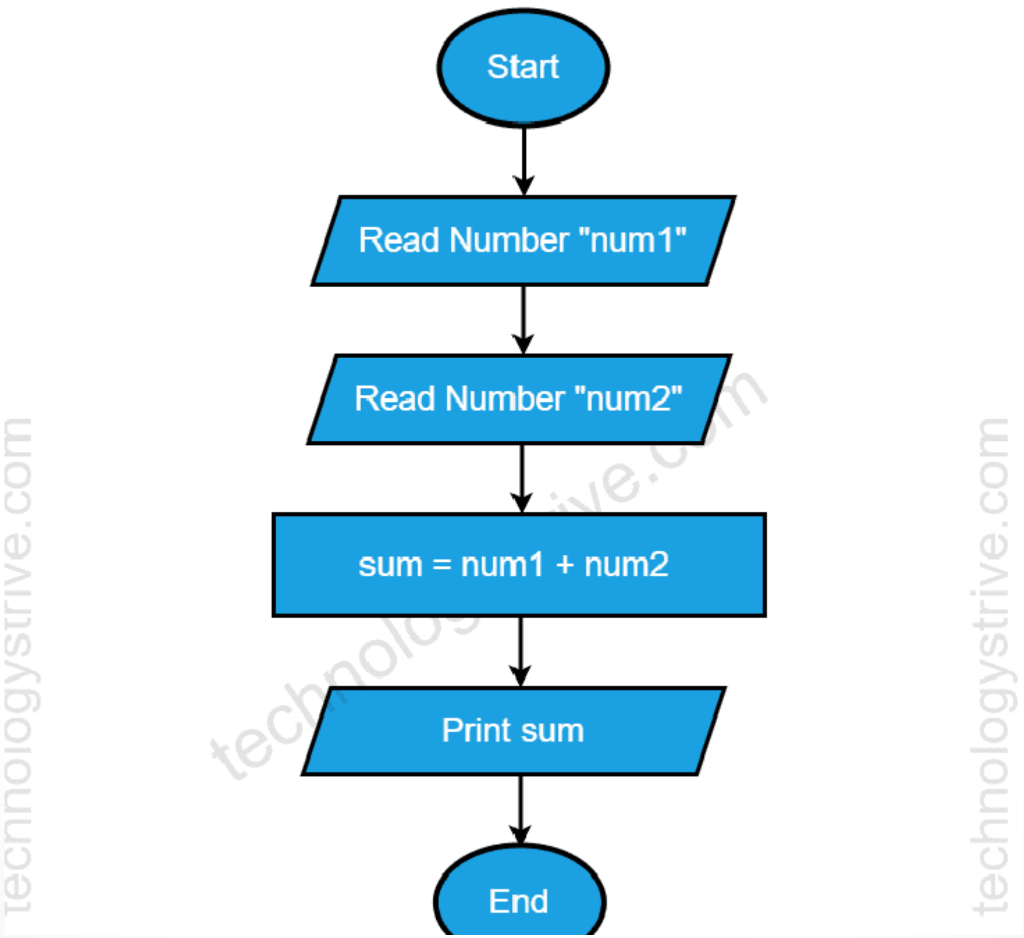
```
#include<stdio.h>

main()

 {

      int a,b; float c;
      printf("Enter any number");
      a=getchar();
      printf("the char is ");
      putchar(a);
      printf("Exhibiting the use of scanf");
     printf("Enter three numbers");
      scanf("%d%d%f",&a,&b,&c);
     printf("%d%d%f",a,b,c);

  }
```

Flowchart & Algorithm

- A Flow chart is a Graphical representation of an Algorithm or a portion of an Algorithm. Flow charts are drawn using certain special purpose symbols such as Rectangles, Diamonds, Ovals and small circles. These symbols are connected by arrows called flow lines.
  - (or)
- The diagrammatic representation of way to solve the given problem is called flow chart.
-
- **The following are the most common symbols used in Drawing flowcharts:**

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectagle represents a process
	Decision	A diamond indicates a decision

Example -Flowchart



Algorithm

- Algorithm is a procedure for solving Mathematical functions
- It is step by step by Step Approach for solving problem.

Example of Algorithm

- STEP 1: Start the program.
- STEP 2: Read the values of 'a'&'b'.
- STEP 3: Compute the sum of the entered numbers 'a','b',c=a+b.
- STEP 4: Print the value of 'c'.
- STEP 5: Stop the program.

Important questions Unit-1

- What is Computer System. Explain computer Hardware with diagram
- List and Explain Different types of Software.
- List out and explain different Programming Languages.
- Explain Basic Structure of C Program with Example
- Explain Different Data types with example
- Explain Formatted input and formatted output
- Write a c program to calculate sum of 3 numbers
- Write a c Program to for simple calculator
- Write a c program for quadratic equation
- Write a program to find area and circumference of the circle.

Case Study

- Profit and loss for small vender shop
- Scenario: Daily income basis

- Thank You

Unit-II

Operators & Expressions

OPERATORS :

- An operator is a symbol that tells the compiler to perform certain mathematical or logical manipulations. They form expressions.
- C operators can be classified as

1.Arithmetic operators

2.Relational operators

3.Logical operators

4.Assignment operators

5.Increment or Decrement operators

6.Conditional operator

7.Bit wise operators

8.Special operators

ARITHMETIC OPERATORS

All basic arithmetic operators are present in C.

operator	meaning
+	add
-	subtract
*	multiplication
/	division
%	modulo division(remainder)

- An arithmetic operation involving only real operands(or integer operands) is called real arithmetic(or integer arithmetic). If a combination of arithmetic and real is called mixed mode arithmetic.

RELATIONAL OPERATORS

- We often compare two quantities and depending on their relation take certain decisions for that comparison we use relational operators.

- Operator Meaning

Operator	Meaning
<	is less than
>	Is greater than
<=	is less than or equal to
>=	is greater than or equal to
==	is equal to
!=	is not equal to

LOGICAL OPERATORS

- An expression of this kind which combines two or more relational expressions is termed as a logical expressions or a compound relational expression. The operators and truth values are

op-1	op-2	op-1 && op-2	op-1 || op-2
non-zero	non-zero	1	1
non-zero	0	0	1
0	non-zero	0	1
0	0	0	0

ASSIGNMENT OPERATORS

- They are used to assign the result of an expression to a variable. The assignment operator is '='.
- v op=exp
- v is variable
- op binary operator
- exp expression
- op= short hand assignment operator

short hand assignment operators

- use of simple assignment operators.

use of short hand assignment operators

- a=a+1 a+=1
- a=a-1 a-=1
- a=a%b a%=b

INCREMENT AND DECREMENT OPERATORS

- ++ and == are called increment and decrement operators used to add or subtract. Both are unary and as follows
- ++m or m++
- --m or m--

- The difference between ++m and m++ is
- if m=5;

  y=++m then it is equal to m=5;

- m++;y=m;
- if m=5;

  y=m++ then it is equal to m=5;
- y=m;m++;

CONDITIONAL OPERATOR

- A ternary operator pair "?:" is available in C to construct conditional expressions of the form

Syntax:

- exp1 ? exp2 : exp3;

It work as

- if exp1 is true then exp2 else exp3

BIT WISE OPERATORS

- C supports special operators known as bit wise operators for manipulation of data at bit level. They are not applied to float or double.

- operator meaning

operator	meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
<<	left shift
>>	right shift
~	one's complement

SPECIAL OPERATORS

- These operators which do not fit in any of the above classification are
- ,(comma), sizeof, Pointer operators(& and *) and member selection operators (. and ->). The comma operator is used to link related expressions together.
- sizeof operator is used to know the sizeof operand.

programs to exhibit the use of operators

```
#include<stdio.h>
int   main()
  {
   int sum, mul, modu;
   float sub, divi;
  int i,j;
  float l, m;
  printf("Enter two integers ");
  scanf("%d%d",&i,&j);
  printf("Enter two real numbers");
 scanf("%f%f",&l,&m);
 sum=i+j; mul=i*j;
 modu=i%j;
 sub=l-m;
 divi=l/m;
 printf("sum is %d", sum);
 printf("mul is %d", mul);
 printf("Remainder is %d", modu);
 printf("subtraction of float is %f", sub);
 printf("division of float is %f", divi);
return 0;
}
```

program to implement relational and logical

- ```
  #include<stdio.h>
    int main()

    {
       int i, j, k;
      printf("Enter any three numbers ");
     scanf("%d%d%d", &i, &j, &k);
      if((i<j)&&(j<k))
                   printf("k is largest");
      else if i<j || j>k
       {
         if i<j && j >k
         printf("j is largest");
     else
        printf("j is not largest of all");
    }
     }
  ```
  ```

program to implement increment and decrement operators

```
• #include<stdio.h>
  Int main()
 {
   int i;
    printf("Enter a number");
    scanf("%d", &i);
   i++;
   printf("after incrementing %d ", i);
    i--;
   printf("after decrement %d", i);
   return 0;
  }
```

program using ternary operator and assignment

- ```
  #include<stdio.h>
  int main()

  {

    int i,j,large;

    printf("Enter two numbers ");
    scanf("%d%d",&i,&j);
   large=(i>j)?i:j;

    printf("largest of two is %d",large);

  }
  ```
  ```

DECISION MAKING

- We have a number of situations where we may have to change the order of execution of statements based on certain conditions or repeat a group of statements until certain specified conditions are met.

The if statement is a two way decision statement and is used in conjunction with an expression. It takes the following form

```
If(test expression)
```

- If the test expression is true then the statement block after if is executed otherwise it is not executed

Syntax:

```
if (test expression)

  {

  statement block;
 }
 statement–x ;
```

Example Program

- ```
  #include<stdio.h>
   int  main()
  {
    int a,b;
    printf("Enter two numbers");
    scanf("%d%d",&a,&b):
    if a>b
   printf(" a is greater");
   if b>a
  printf("b is greater");
   return 0;
   }
  ```
  ```

The if –else statement:

- If your have another set of statement to be executed if condition is false then if-else is

  used

Syntax:

```
    if (test expression)
   {
    statement block1;
  }
 else
  {
   statement block2;
   }
 statement –x ;
```

program for if-else

- ```
  #include<stdio.h>
  int main()
  {
   int a,b;
  printf(“Enter two numbers”);
  scanf(“%d%d”,&a,&b):
   if a>b
  printf(“ a is greater”)
  else
  printf(“b is greater”);
  return 0;
  }
  ```
  ```

Nesting of if..else statement :

```
•if(text cond1)
        {

             if (test expression2

                 {

                  statement block1;

                  }

        else
                 {

                   statement block 2;

                }

          }
      else

           {

         statement block2;

         }

        statement-x ;
```

THE SWITCH STATEMENT

- If for suppose we have more than one valid choices to choose from then switch statement in place of if statements.

Syntax:

```
switch(expression)
 {.
 case value-1:
                    block-1
                    break;
case value-2:
                    block-2
                    break;
  --------
default:
                  default block;
                 break;
}
 statement–x
```

Example Program for Switch

```
#include<stdio.h>
main()
 {
 int marks,index;
char grade[10];
printf("Enter your marks");
scanf("%d",&marks);
index=marks/10;
 switch(index)
{
 case 10 :
 case 9:
 case 8:
 case 7:
 case 6: grade="first";       break;
case 5 : grade="second";     break;
case 4 : grade="third";      break;
default : grade ="fail";     break;
}

printf("%s",grade);

}
```

Repetition

- **LOOPING :** Some times we require a set of statements to be executed repeatedly until a condition is met.

  We have two types of looping structures.

  1.One in which condition is tested before entering the statement block called entry control.

  2.The other in which condition is checked at exit called exit controlled loop.

WHILE STATEMENT

While(test condition)

{

body of the loop

}

It is an entry controlled loop. The condition is evaluated and if it is true then body of loop is executed. After execution of body the condition is once again evaluated and if is true body is executed once again. This goes on until test condition becomes false

Example program-While

```
#include<stdio.h>
 main()
{

int count,n;
 float x,y;
printf("Enter the values of x and n");

scanf("%f%d",&x,&n);
 y=1.0;
count=1; while(count<=n)
{

y=y*x;

count++;
}

printf("x=%f; n=%d; x to power n = %f",x,n,y);

}
```

DO WHILE STATEMENT

The while loop does not allow body to be executed if test condition is false. The do while is an exit controlled loop and its body is executed at least once.

Syntax:

```
do
{
body
}while(test condition);
```

Example Program-Do While

```
// Program to add numbers until the user enters zero
#include <stdio.h>
int main() {
  double number, sum = 0;
  // the body of the loop is executed at least once
  do {
    printf("Enter a number: ");
    scanf("%lf", &number);
    sum += number;
  }
  while(number != 0.0);
  printf("Sum = %.2lf",sum);
  return 0;
}
```

Output:

```
Enter a number: 1.5
Enter a number: 2.4
Enter a number: -3.4
Enter a number: 4.2
Enter a number: 0
Sum = 4.70
```

THE FOR LOOP

It is also an entry control loop that provides a more concise structure

Syntax:

```
for(initialization; test control; increment)
{

body of loop

}
```

Example –For Loop

```
#include<stdio.h>
main()
{

long int p; int n; double q;
printf("2 to power n ");
p=1;
for(n=0;n<21;++n)
 {

if(n==0)

p=1;
 else
 p=p*2; q=1.0/(double)p;
printf("%101d%10d",p,n);
 }

}
```

Expressions

- An Expression is a sequence of operands and operators.
- A simple Expression contains only one operator.
- Ex:2+3=5
- A complex Expression contains more than one operator.
- Ex:2+5*7

UNCONDITIONAL STATEMENTS

BREAK STATEMENT:

This is a simple statement. It only makes sense if it occurs in the body of a switch, do, while or for statement. When it is executed the control of flow jumps to the statement immediately following the body of the statement containing the break. Its use is widespread in switch statements, where it is more or less essential to get the control .

Example-Break

```
#include <stdio.h>
#include <stdlib.h>
main(){
int i;

for(i = 0; i < 10000; i++){
if(getchar() == 's')
 break;
printf("%d\n", i);
} exit(EXIT_SUCCESS);
}
```

- It reads a single character from the program's input before printing the next in a sequence of numbers.
- If you want to exit from more than one level of loop, the break is the wrong thing to use.

Postfix Expression

- *The expression of the form “a b operator” (ab+) i.e., when a pair of operands is followed by an operator.*
- **Examples:**
- **1)Input**: str = “2 3 1 * + 9 -“
  Output: -4
  Explanation: If the expression is converted into an infix expression, it will be
- 2 + (3 * 1) – 9 = 5 – 9 = -4.
- ***2)Input****: str = “100 200 + 2 / 5 * 7 +”*
  Output*: 757*
- *Expalnation:*100 + 200 = 300
  300 / 2 = 150
  150 * 5 = 750
  750 + 7 = 757

Prefix Expression

- To evaluate a postfix expression we can use a stack.
- Iterate the expression from left to right and keep on storing the operands into a stack. Once an operator is received, pop the two topmost elements and evaluate them and push the result in the stack again.
- Prefix and Postfix expressions can be evaluated faster than an infix expression. This is because we don’t need to process any brackets or follow operator precedence rule. In postfix and prefix expressions which ever operator comes before will be evaluated first, irrespective of its priority. Also, there are no brackets in these expressions. As long as we can guarantee that a valid prefix or postfix expression is used, it can be evaluated with correctness.

Example

- Input : -+8/632
- 6/3 + 8 - 2
- Output : 8

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(type){list}	Compound literal(C99)	
2	++ --	Prefix increment and decrement[note 1]	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of[note 2]	
	_Alignof	Alignment requirement(C11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10	|	Bitwise OR (inclusive or)	
11	&&	Logical AND	
12	||	Logical OR	
13	?:	Ternary conditional[note 3]	Right-to-left
14	=	Simple assignment	

Evaluate Expression

- 1.)x=3*4+5*6
- First, we perform the multiplications: 3×4=12
- 3×4=12
- 5×6=30
- 5×6=30
- Next, we perform the additions: 12+30=42   12+30=42
- So, �=42*x*=42.

- 2)x=3*(4+5)*6
- First, we perform the operation inside the parentheses: 4+5=94+5=9
- Next, we multiply: 3×9=273×9=27
- Finally, we multiply by 6: 27×6=16227×6=162
- So, �=162*x*=162.

- 3)x=3*4%5/2
- 4)x=3*(4%5)/2

- 5)x=3*((4%5)/2)
- Calculate 4mod 54mod5, which is the remainder of the division of 4 by 5: 4mod 5=44mod5=4
- Divide the result by 2: 4/2=24/2=2
- Multiply the result by 3: 3×2=63×2=6
- So, �=6*x*=6.

- 6)x=17-8/4*2+3-++a (assume a=6)

For given values int a=0,b=1,c=-1
float x=2.5,y=0.0 Evaluate following expressions

- **1)a+=b-=c*10**

- Ans evaluates to $-1\times10=-10-1\times10=-10$
- $b-=c\times10$ is equivalent to $b=b-(-10)$, which simplifies to $b=b+10$. Since b was 1, it becomes $1+10=11b=1+10=11$.

  $+=b$ is equivalent to $a=a+b$, so 0+11=11 therefore after evaluation a=11,b=11mc=-1

- **2)-a*(5+b)/2-c++*b**

a evaluates to 00 because a is 0

$5+b$ evaluates to $5+1=65+1=6$.

$-a\times(5+b)$ evaluates to $0\times6=00\times6=0$.

0/20/2 evaluates to 00.

c++ increments c by 11 (post-increment), so c becomes 00 after this operation.

$c\times b$ evaluates to $0\times1=00\times1=0$.

Subtracting 00 from 00 (from step 4) gives 00.

Therefore, the expression $-a\times(5+b)/2-c{+}{+}\times b$ evaluates to 00.

Contd..

- **3)a*b*c**

Ans:

0*1*-1=0

- **4)(x>y)+!a||c++**

Ans: int a = 0

int b = 1

int c = -1

float x = 2.5

float y = 0.0

(x > y) evaluates to true since 2.5 is greater than 0.0.

!a evaluates to !0, which is true because ! negates the value.

c++ increments c by 1, so c becomes 0 after this operation.

Now we have true from step 1, true from step 2, and the incremented value of c (which is 0).

true is typically represented as 1 in C-like languages, so we have 1 + 1 + 0.

1 + 1 + 0 evaluates to 2.

Therefore, the expression (x > y) + !a || c++ evaluates to 2 given the provided values

Contd..

- 5)a<b&&c<b

Ans:0<1 && -1<1   true && true=True=1

True=1

- 6)b+c||!a
- Ans:1+(-1)||!1   1-1||0   0||0 =>0
- 7)x*5&&5||(b/c)
- Ans:2.5*5 && 5||(1/-1)
-         12.5 && 5||0
-          True||0   -> true

Contd..

- 8)a<=10&&x>=1&&b
- Ans:
- 9)!x||!c||b+c
- Ans:
- 10)x*y<a+b||c
- Ans:2.5*0.0<0+1||-1
- 0<1||-1   ->TRUE||-1
- 1||-1

Type Conversion

- Type conversion in C is the process of converting one data type to another. The type conversion is only performed to those data types where conversion is possible. Type conversion is performed by a compiler. In type conversion, the destination data type can't be smaller than the source data type. Type conversion is done at compile time and it is also called widening conversion because the destination data type can't be smaller than the source data type. ***There are two types of Conversion.***
- ***Implicit Type Conversion***
- ***Explicit Type Conversion***

Implicit Type Conversion

- Also known as ‘automatic type conversion’.
- A. Done by the compiler on its own, without any external trigger from the user.
- B. Generally takes place when in an expression more than one data type is present. In such conditions type conversion (type promotion) takes place to avoid loss of data.

Example of Implicit type Conversion

```
#include <stdio.h>
int main()
{
   int x = 10; // integer x
   char y = 'a'; // character c

   // y implicitly converted to int. ASCII
   // value of 'a' is 97
   x = x + y;

   // x is implicitly converted to float
   float z = x + 1.0;

   printf("x = %d, z = %f", x, z);
   return 0;
}
```

Output:

x = 107, z = 108.000000

Explicit Type Conversion

- This process is also called type casting and it is user-defined. Here the user can typecast the result to make it of a particular data type. The syntax in C Programming:

Syntax:

- (type) expression

Type indicated the data type to which the final result is converted.

Example of Explicit Type conversion

```
#include<stdio.h>

int main()
{
    double x = 1.2;

    // Explicit conversion from double to int
    int sum = (int)x + 1;

    printf("sum = %d", sum);

    return 0;
}
```

Output: sum = 2

CONTINUE STATEMENT:

- This statement has only a limited number of uses. The rules for its use are the same as for break, with the exception that it doesn't apply to switch statements. Executing a continue starts the next iteration of the smallest enclosing do, while or for statement immediately. The use of continue is largely restricted to the top of loops, where a decision has to be made whether or not to execute the rest of the body of the loop. In this example it ensures that division by zero (which gives undefined behavior) doesn't happen

Example -Continue

```
#include <stdio.h>
 #include <stdlib.h>
main(){
int i;

for(i = -10; i < 10; i++)
{ if(i == 0)
continue;
 printf("%f\n", 15.0/i);
/*
* Lots of other statements .....
*/
} exit(EXIT_SUCCESS);

}
```

Contd..

- The continue can be used in other parts of a loop, too, where it may occasionally help to simplify the logic of the code and improve readability. continue has no special meaning to a switch statement, where break does have. Inside a switch, continue is only valid if there is a loop that encloses the switch, in which case the next iteration of the loop will be started.

GOTO AND LABELS

In C, it is used to escape from multiple nested loops, or to go to an error handling exit at the end of a function.

You will need a *label* when you use a goto;

this example shows both.

```
goto L1;
/* whatever you like here */  L1: /* anything else */
```

A label is an identifier followed by a colon. Labels have their own „name space‟ so they can't clash with the names of variables or functions. The name space only exists for the function containing the label, so label names can be re-used in different functions. The label can be used before it is declared, too, simply by mentioning it in a goto statement.

Labels must be part of a full statement, even if it's an empty one. This usually only matters when you're trying to put a label at the end of a compound statement—like this.

```
label_at_end: ; /* empty statement */
}
```

The goto works in an obvious way, jumping to the labelled statements. Because the name of the label is only visible inside its own function, you can't jump from one function to another one.

It's hard to give rigid rules about the use of gotos but, as with the do, continue and the break (except in switch statements), over-use should be avoided. More than one goto every 3–5 functions is a symptom that should be viewed with deep suspicion.

Important Questions

1. List out all operators in C. Write a program using Switch include all operators.
2. Analyze the following code write the output.

```
#include<stdio.h>
void main()
{
printf("7 && 0=%d\n",7&&0);
printf("7 && 0=%d\n",7||0);
printf("!0=%d\n",!0);
}
```

3. What are Infix, Prefix, Postfix Expression.

Solve the following prefix and Postfix Expressions. For given values int a=0,b=1,c=-1 float x=2.5,y=0.0 Evaluate following expressions.

1.(x>y) +! a||c++

2. X*5&&5||(b/c)

3.!x||!c||b+c

4.a<=10&&x>=1&&b

5.-a*(5+b)/2-c++*b

6.x*y<a+b||c

4.What are Implicit and Explicit Type Conversion? Explain with example.

5.List out and explain the different types of Loops in C

6.Write a C Program to print a given integer number is palindrome or not

7.Discuss break and continue statements with suitable examples.

8.Program to calculate the total sales given the unit price, quantity, discount and tax rate

9.Write a C-Program to calculate a student's average score for a course with 4 quizzes, 2 midterms and a final. The quizzes are weighted 30%, the midterms 40% and the final 30%.

10.

11.Write C-Program to evaluate two complex expressions.

12.Write C-Program to read a test score, calculate the grade for the score and print the grade

13.Write a C Program to print Square of Index and Print it.

Case study

Case Study: To display Students grade

Scenario: Only Student Grade List of Ist year students of Cyber Security Branch

Case Study:Set Password

Scenario:After Setting Password for wrong Entry

Ex:0 and o